Practical Work 2 — Software Development Life Cycle and Requirements

Design of Software System · Turin Polytechnic University in Tashkent

Instructions

This practical work covers the material of Lecture 2 up to the properties of a good requirement: the six phases of the software development life cycle, the cost of a late mistake, functional and non-functional requirements, the techniques of collecting requirements, and the six properties that every written requirement must have.

- Work in pairs during the session, but every student submits their own file.
- You may use your lecture slides. You may not copy the answers of another pair.
- Write your answers in one document and submit it as a PDF named `Surname_Name_Prac2.pdf`.
- Tasks 1–4 are done during the session. Task 5 is the main task and may be finished at home.

Total: 20 points. Time in class: 80 minutes.

Task 1 — The six phases (2 points, 10 minutes)

The six phases of the SDLC are given below in the wrong order, together with their outputs, also in the wrong order.

```
PHASES:
   Testing  |  Requirement gathering  |  Deployment and maintenance
   Design   |  Coding                 |  Feasibility study

OUTPUTS:
   Source code and unit tests       |  SRS document signed by the customer
   Test report and list of defects  |  Working system and a user manual
   HLD and LLD documents            |  A report: the project is approved
```

1.1 Write the six phases in the correct order, from 1 to 6.

1.2 Write the correct output next to each phase.

1.3 In one sentence for each: what would go wrong in the project if that phase were skipped?

Task 2 — The cost of a late mistake (2 points, 10 minutes)

In the lecture we saw that the relative cost of fixing one mistake grows in every phase: 1 in requirements, 5 in design, 10 in coding, 50 in testing and about 150 after release.

A team is building an online shop. The customer said "the delivery is free for orders over 500 000 soum", but the analyst wrote "the delivery is free for orders over 5 000 000 soum" in the requirements document.

2.1 Name three different things that must be changed if this mistake is found during testing, and not during the requirements phase.

2.2 Name one thing that can be damaged if the mistake is found by the customers themselves, and that money cannot repair.

2.3 Which phase of the SDLC would have caught this mistake at the cost of one unit, and how exactly?

Task 3 — Functional or non-functional? (4 points, 20 minutes)

Read the case below. It is used in tasks 3, 4 and 5.

Case study: the TTPU cafeteria pre-order system.

Students lose too much time standing in the queue of the university cafeteria during the 20-minute break. The university wants a small web application. A student opens the menu of the day, selects the dishes, pays with a card or chooses to pay at the counter, and receives an order number. The kitchen sees the new orders on a screen and prepares them. When the order is ready, the student gets a notification and comes to a separate desk to take the food. The cafeteria manager wants to see how many portions of each dish were sold, so that less food is thrown away in the evening. The system will be used by about 2 000 students, and almost all of them open it in the same ten minutes before the break.

For each of the twelve statements below, write **F** if it is a functional requirement and **N** if it is non-functional. For every statement you mark **N**, also write which quality it belongs to: performance, security, usability, reliability, portability or maintainability.

```
1.  The system shall show the menu of the current day to a logged-in student.
2.  The menu page shall load in under 2 seconds when 1 000 students open it at once.
3.  The system shall allow a student to pay the order with a bank card.
4.  Card numbers shall never be stored in the database of the cafeteria.
5.  The system shall send a notification to the student when the order is ready.
6.  The interface shall be available in Uzbek, Russian and English.
7.  The system shall give every new order a number that is unique for that day.
8.  The system shall work in Chrome, Safari and on the university tablets.
9.  The kitchen screen shall show all orders that are not yet prepared.
10. The system shall not be unavailable for more than 10 minutes during a working day.
11. The manager shall be able to see the number of portions sold for each dish of the day.
12. A new dish category shall be added without changing the code of the payment module.
```

Task 4 — Rewrite the bad requirements (4 points, 20 minutes)

The six statements below were written by a junior analyst for the same cafeteria system. None of them can be tested. Rewrite each of them so that it satisfies the six properties from the lecture: clear, testable, complete, consistent, necessary and traceable.

Rules for your answer: use the word **shall**, put a **number** in the sentence whenever it is possible, give every requirement an **ID** (FR-01, NFR-01, …), and add one line describing **how you would test it**.

```
a)  The application must be fast.
b)  The menu should be easy to use.
c)  The system must support a lot of students.
d)  Payments must be secure.
e)  The kitchen should probably see the orders quickly.
f)  Reports must be useful for the manager.
```

Example of the expected form of one answer:

```
FR-07  The system shall show the order on the kitchen screen within 5 seconds
       after the payment is confirmed.
Test:  Pay for an order, start a timer, and measure the delay until the order
       appears on the kitchen screen. Repeat 10 times; every measurement must
       be under 5 seconds.
```

Task 5 — Write the requirements yourself (6 points, main task)

You are the analyst of the cafeteria project. Using the case study of task 3, produce the following four parts.

5.1 Requirements list. Write at least **eight functional** and **five non-functional** requirements. Every requirement must have an ID, the word "shall", and a number wherever a number is possible. Your five non-functional requirements must cover at least four different qualities.

5.2 Collection techniques. For each of the five techniques from the lecture — interview, questionnaire, observation, document analysis, prototyping — write one line: from whom, and what exactly you would learn with it in this project. Then name the technique you would use first, and why.

5.3 Questions the customer did not answer. The case study does not say everything. Write five questions you would ask the university before you start the design. For example: what happens to an order that nobody comes to take?

5.4 Self-check table. Take any three of your own requirements and check them against the six properties. Use this table:

ID	Clear	Testable	Complete	Consistent	Necessary	Traceable
FR-01						
FR-02						
NFR-01						

Write "yes" or "no" in every cell. If you write "no" anywhere, correct the requirement below the table and explain in one sentence what you changed.

Task 6 — Bonus: find the contradiction (2 points)

A second analyst joined the project and added the three requirements below to the same document.

```
FR-20   The student shall be able to cancel an order at any moment before taking the food.
FR-21   The kitchen shall start preparing an order immediately after the payment is confirmed.
NFR-09  The cafeteria shall throw away no food at the end of the day.
```

6.1 Which two of these requirements contradict each other, and why?

6.2 Which of the three is not a requirement of the software at all? Explain what it really is.

6.3 Rewrite the contradicting pair so that both can be implemented.

Grading

Task	What is checked	Points
1	Correct order of the phases, correct outputs, sensible consequences	2
2	Three real consequences, one non-repairable damage, the right phase named	2
3	Correct F/N for 12 statements and the right quality for every N	4
4	Six requirements rewritten with ID, "shall", a number and a test line	4
5	13 requirements, five techniques, five questions, the self-check table	6
6	Bonus: the contradiction found and explained, the pair rewritten	2
	Total	**20**

Submission. One PDF file named `Surname_Name_Prac2.pdf`, sent on Telegram to @ethz_uz before the deadline announced in class. Work sent after the deadline is not accepted. These points count towards your intermediate control (IC) score.